

TASK 1 — Create Main Loop and Display Prompt

```
krishnasree08@krishnasree08:~/practical_week2/OSSP/src$ cd ~/OSSP
krishnasree08@krishnasree08:~/OSSP$ pwd
/home/krishnasree08/OSSP
krishnasree08@krishnasree08:~/OSSP$ cd src
krishnasree08@krishnasree08:~/OSSP/src$ nano main.c
krishnasree08@krishnasree08:~/OSSP/src$ gcc main.c -o main
krishnasree08@krishnasree08:~/OSSP/src$ ./main
myshell> hello
You entered: hello
myshell> ls
You entered: ls
myshell> exit
Exiting shell...
```

```
#include <stdio.h>
#include <string.h>

int main()
{
    char command[100];

    while (1)
    {
        printf("myshell> ");
        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        printf("You entered: %s\n", command);
    }

    return 0;
}
```

Summary

A basic interactive shell loop was created. The program displays a prompt, accepts user input, processes commands, and exits when the user enters exit. The task focuses on creating the main loop, displaying the prompt, reading input, and handling exit conditions.

Result

Thus, the basic interactive shell loop was successfully implemented and tested.

TASK 2 — Copy File1.txt to File2.txt Using System Calls

```
krishnasree08@krishnasree08:~/OSSP/src$ cd ~/OSSP
krishnasree08@krishnasree08:~/OSSP$ echo "Hello, this is File1." > File1
krishnasree08@krishnasree08:~/OSSP$ cat File1.txt
Hello, this is File1.
krishnasree08@krishnasree08:~/OSSP$ cd src
nano filecopy.c
krishnasree08@krishnasree08:~/OSSP/src$ gcc filecopy.c -o filecopy
krishnasree08@krishnasree08:~/OSSP/src$ ./filecopy
File copied successfully.
krishnasree08@krishnasree08:~/OSSP/src$ cd ..
krishnasree08@krishnasree08:~/OSSP$ cat File2.txt
Hello, this is File1.
```

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd1, fd2;
    char buffer[100];
    int n;

    fd1 = open("../File1.txt", O_RDONLY);

    if (fd1 < 0)
    {
        perror("Error opening File1.txt");
        return 1;
    }

    fd2 = open("../File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

    if (fd2 < 0)
    {
        perror("Error opening File2.txt");
        close(fd1);
        return 1;
    }

    while ((n = read(fd1, buffer, sizeof(buffer))) > 0)
    {
        write(fd2, buffer, n);
    }

    close(fd1);
    close(fd2);

    printf("File copied successfully.\n");
}

```

Summary:

A C program was developed to copy the contents of File1.txt to File2.txt using the open(), read(), write(), and close() system calls.

Result:

Thus, the contents of File1.txt were successfully copied to File2.txt using Linux file system calls.

TASK 3 — Capture Keyboard Input and Handle Backspace

```
krishnasree08@krishnasree08:~/OSSP/src$ cd ~/OSSP/src
nano input.c
krishnasree08@krishnasree08:~/OSSP/src$ gcc input.c -o input
krishnasree08@krishnasree08:~/OSSP/src$ ./input
hello
hellomyshell>
You entered: hello
krishnasree08@krishnasree08:~/OSSP/src$ |
```

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    char buffer[100];
    int index = 0;
    char ch;

    printf("myshell> ");

    while (1)
    {
        read(STDIN_FILENO, &ch, 1);

        if (ch == '\n')
        {
            buffer[index] = '\0';
            printf("\nYou entered: %s\n", buffer);
            break;
        }
        else if (ch == 127)
        {
            if (index > 0)
            {
                index--;
                printf("\b \b");
            }
        }
        else
        {
            if (index < 99)
            {
                buffer[index++] = ch;
                write(STDOUT_FILENO, &ch, 1);
            }
        }
    }
}
```

Summary

Keyboard input was captured character-by-character using the `read()` system call. The program manages an input buffer, handles backspace, processes the Enter key, and supports multi-character commands.

Result

Thus, keyboard input and basic editing functionality were successfully implemented.

TASK 4 — Command History

```
krishnasree08@krishnasree08:~/OSSP/src$ nano history.c
krishnasree08@krishnasree08:~/OSSP/src$ gcc history.c -o history
krishnasree08@krishnasree08:~/OSSP/src$ ./history
myshell> ls
Command stored: ls
myshell> pwd
Command stored: pwd
myshell> date
Command stored: date
myshell> ecit
Command stored: ecit
myshell> exit

Command History:
1: ls
2: pwd
3: date
4: ecit
```

```

#include <stdio.h>
#include <string.h>

int main()
{
    char history[10][100];
    int count = 0;
    char command[100];

    while (count < 10)
    {
        printf("myshell> ");
        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        strcpy(history[count], command);
        count++;

        printf("Command stored: %s\n", command);
    }

    printf("\nCommand History:\n");

    for (int i = 0; i < count; i++)
    {
        printf("%d: %s\n", i + 1, history[i]);
    }

    return 0;
}

```

Summary

A command history mechanism was implemented to store previously entered commands and display them when required. The task focuses on command history storage, navigation, input-buffer updates, and recall functionality.

Result

Thus, command history storage and recall functionality were successfully implemented and tested.

TASK 5 — Dynamic Memory and Valgrind

```
krishnasree08@krishnasree08:~/OSSP/src$ sudo apt update
[sudo] password for krishnasree08:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Package
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-e
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Compone
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Pa
Get:11 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Componen
Get:12 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Pack
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Co
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted amd64
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/universe Translatio
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Comp
Get:17 http://security.ubuntu.com/ubuntu noble-security/restricted Transl
Get:18 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Pa
Get:19 http://archive.ubuntu.com/ubuntu noble-updates/restricted Translat
Get:20 http://archive.ubuntu.com/ubuntu noble-updates/multiverse Translat
Get:21 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Co
Get:22 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Compon
Get:23 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Co
Fetched 10.3 MB in 11s (913 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
86 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
krishnasree08@krishnasree08:~/OSSP/src$ sudo apt install valgrind
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

```
Processing triggers for libc-bin (2.35-0ubuntu0.0) ...
krishnasree08@krishnasree08:~/OSSP/src$ nano memory.c
krishnasree08@krishnasree08:~/OSSP/src$ gcc -g memory.c -o memory
krishnasree08@krishnasree08:~/OSSP/src$ ./memory
Enter number of elements: 5
Array elements:
1 2 3 4 5
krishnasree08@krishnasree08:~/OSSP/src$ valgrind --leak-check=full ./mem
==1588== Memcheck, a memory error detector
==1588== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1588== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright i
==1588== Command: ./memory
==1588==
Enter number of elements: 6
Array elements:
1 2 3 4 5 6
==1588==
==1588== HEAP SUMMARY:
==1588==       in use at exit: 0 bytes in 0 blocks
==1588==   total heap usage: 3 allocs, 3 frees, 2,072 bytes allocated
==1588==
==1588== All heap blocks were freed -- no leaks are possible
==1588==
==1588== For lists of detected and suppressed errors, rerun with: -s
==1588== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
krishnasree08@krishnasree08:~/OSSP/src$ |
```

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int *arr = malloc(n * sizeof(int));

    if (arr == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < n; i++)
    {
        arr[i] = i + 1;
    }

    printf("Array elements:\n");

    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n");

    free(arr);

    return 0;
}
```

Summary

Dynamic memory allocation was implemented using malloc() and memory was released using free(). The program was validated using Valgrind to check for memory leaks and ensure that dynamically allocated memory was properly released.

Result

Thus, dynamic memory allocation and deallocation were successfully implemented and validated using Valgrind.

